

Database Systems

Nitin Awathare

Department of Computer Science and Engineering,
IIT Jodhpur

Course Structure

- Minor test - 30%
- Major test - 40%
- Assignment(2) - 20%
- Quiz (2) - 10%

Database use cases

Organizational Data Management Use Cases

- **Commercial Operations:** maintenance of customer records, product information, and transaction histories
- **Financial Administration:** tracking of income and expenditures, asset control, and accounting documentation
- **Workforce Management:** storage and processing of employee details, compensation data, and statutory payroll information

Industrial and Production Systems

- Coordination of manufacturing activities, inventory control, order fulfillment, and end-to-end supply logistics

Financial and Banking Services

- Management of client profiles, deposit and loan accounts, and daily transactional activity
- Processing and auditing of card-based payments
- Execution and recording of trades involving financial assets, supported by real-time market information

Educational Information Systems

- Handling of student enrollment, academic records, and evaluation outcomes

Issues with the ancient systems

In early computing systems, databases were implemented as collections of independent files managed directly by application programs. This design resulted in several fundamental limitations:

- **Repetition and inconsistency of data:** the same information was duplicated across multiple files and formats, often leading to mismatched or outdated values
- **Poor data accessibility:** retrieving information required custom programs for each specific task
- **Lack of flexibility:** even simple changes or new queries demanded rewriting application code
- **Data fragmentation:** information was scattered across separate files with no unified structure
- **Weak enforcement of rules:** data constraints (such as ensuring a positive account balance) were embedded within application logic rather than defined centrally
- **Limited adaptability:** introducing new constraints or modifying existing ones was difficult and error-prone

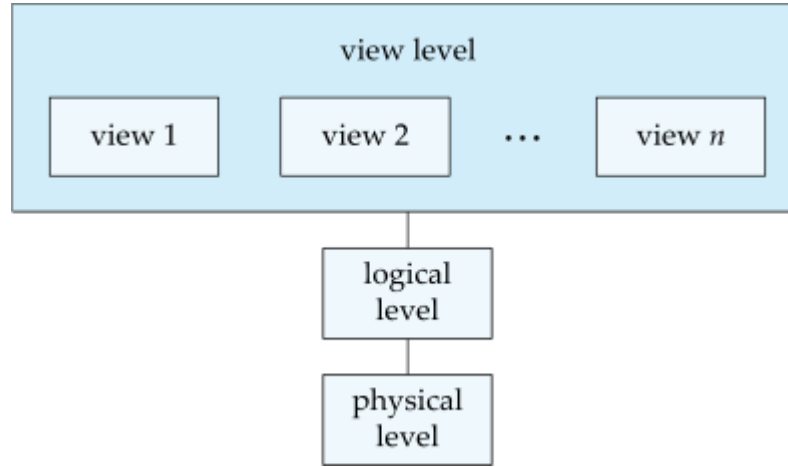
Issues with the ancient systems (Cont.)

Atomic updates: system failures can interrupt operations, leaving data only partially modified; for example, a fund transfer must execute fully or be rolled back entirely.

Simultaneous user access: parallel access improves performance, but without proper control it can cause conflicting updates, such as multiple users withdrawing from the same balance concurrently.

Access control challenges: restricting users to authorized subsets of data is difficult without built-in security mechanisms.

How database resolve the issue



Physical level: lowest layer, defining how data is physically stored using underlying data structures.

Logical level: specifies the data schema and relationships, independent of storage details, supporting physical data independence.

View level: highest layer, presenting customized subsets of the database for specific users or applications.

Example

```
type instructor = record
  ID          : char(5);
  name        : char(20);
  dept_name   : char(20);
  salary      : numeric(8,2);
end;
```

Physical level: records are mapped to storage blocks; storage details are managed by the DBMS and hidden from programmers.

Logical level: defines record structures and relationships, similar to data types; used by DBAs and developers.

View level: users access limited data through queries or interfaces; views simplify usage and enforce security (e.g., student data without salary details).

Database Schema

Database instance: the actual data stored at a particular point in time.

Database schema: the overall structure of the database; changes infrequently.

Programming analogy: schema $\approx$ variable declarations (types), instance $\approx$ variable values during execution.

Multiple schema levels:

- **Physical schema:** how data is stored on disk.
- **Logical schema:** what data is stored and how it is related.
- **View schemas:** user-specific subsets of the data.

Application dependence: programs rely on the logical schema, not the physical schema.

Physical data independence: changes to physical storage do not require changes to application programs.

Database Models

Data model: a conceptual framework defining data structure, relationships, constraints, and semantics across physical, logical, and view levels.

Relational model: represents data as tables (relations) with rows and columns; record-based and the most widely used model.

Entity–Relationship (ER) model: models data using entities and their relationships; mainly used for conceptual design.

Object-oriented model: extends ER concepts with object-oriented features such as encapsulation and object identity.

Object–relational model: integrates object-oriented features into the relational model; supported by many modern DBMSs.

Semistructured model: supports flexible schemas with varying attributes; commonly represented using XML.

Earlier models: hierarchical and network models were implementation-specific and are largely obsolete today.

Database Languages

Data-Manipulation Language (DML):

- Supports data retrieval, insertion, deletion, and updates
- **Procedural DML:** specifies *what* data is needed and *how* to obtain it
- **Declarative DML:** specifies only *what* data is needed; DBMS decides *how*
- Queries are DML statements used for data retrieval

Data-Definition Language (DDL):

- Defines schema, storage structures, access methods, and constraints
- **Integrity constraints:**
 - Domain constraints (valid attribute values)
 - Referential integrity (consistency across relations)
 - Assertions (general conditions)
 - Authorization rules (access control)
- Metadata is stored in the **data dictionary**

Query Processor

The **query processor** consists of the following components:

- **DDL interpreter:** processes Data Definition Language (DDL) statements and stores schema definitions in the data dictionary.
- **DML compiler:** converts Data Manipulation Language (DML) statements into low-level execution plans for the query evaluation engine; it also performs query optimization by selecting the most efficient plan among alternatives.
- **Query evaluation engine:** executes the optimized execution plan to retrieve or modify data efficiently.

Data Storage and Querying

Storage manager: interfaces applications with disk-resident data, organizes storage to minimize I/O, and translates DML operations into file-system actions.

Query processor: converts high-level queries into optimized low-level execution plans, enabling efficient data access while hiding physical storage details.

Data Storage Manager

The **storage manager** is composed of several core components:

- **Authorization and integrity manager:** controls user access and enforces integrity constraints
- **Transaction manager:** preserves database consistency during failures and coordinates concurrent transactions
- **File manager:** allocates disk space and organizes on-disk data structures
- **Buffer manager:** manages movement of data between disk and main memory, caching frequently accessed data for efficiency

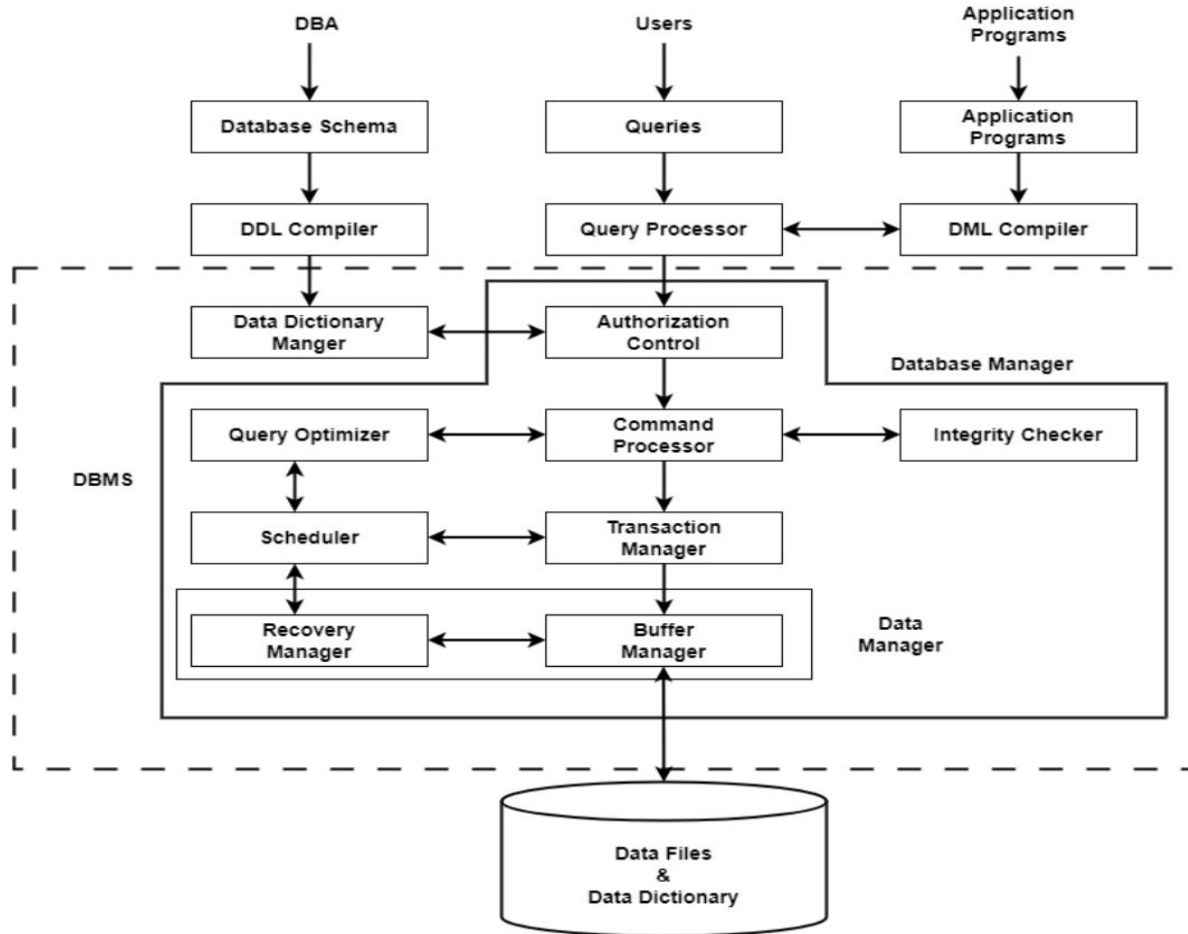
In addition, the storage manager maintains key physical data structures:

- **Data files:** contain the actual database records
- **Data dictionary:** stores metadata, including schema definitions
- **Indices:** Provide fast data access by mapping values to record locations.

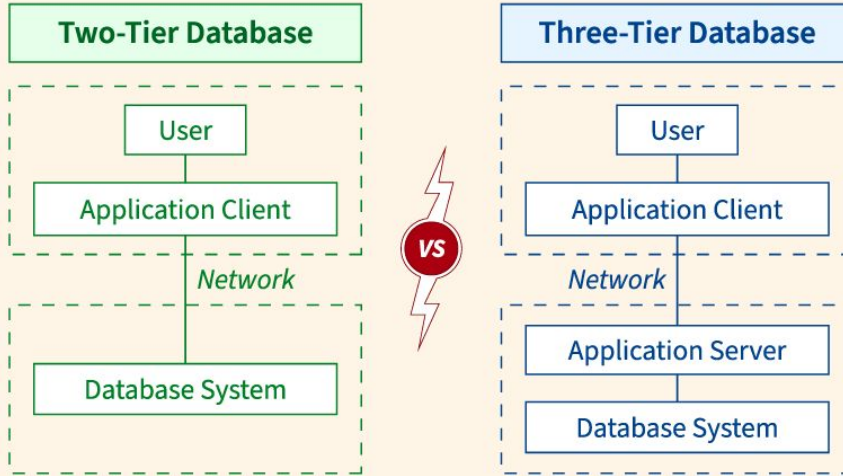
Transaction

- **Logical unit of work:** multiple operations may form a single task (e.g., fund transfer with debit and credit).
- **Atomicity:** all operations of a transaction execute completely or not at all.
- **Consistency:** a transaction preserves database correctness before and after execution.
- **Durability:** once committed, transaction effects persist despite failures.
- **Transaction:** a sequence of operations satisfying **ACID** (Atomicity, Consistency, Isolation, Durability).
- **Programmer's role:** define transactions that move the database between consistent states.
- **System support:**
 - **Recovery manager:** ensures atomicity and durability after failures.
 - **Concurrency-control manager:** preserves consistency during concurrent execution.

Database Architecture



Database Architecture



THANK YOU